

Operating System Structure & Operations



By:

Dr. H. K. Azad

Assistant Professor

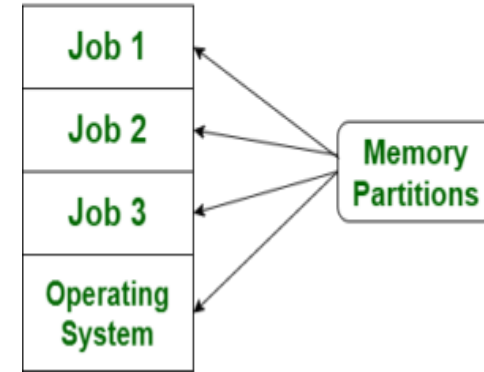
SCOPE

Email: hiteshwarkumar.azad@vit.ac.in

Operating System Structure

- **Multiprogramming** needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so, CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - When it has to wait (for I/O for example), OS switches to another job

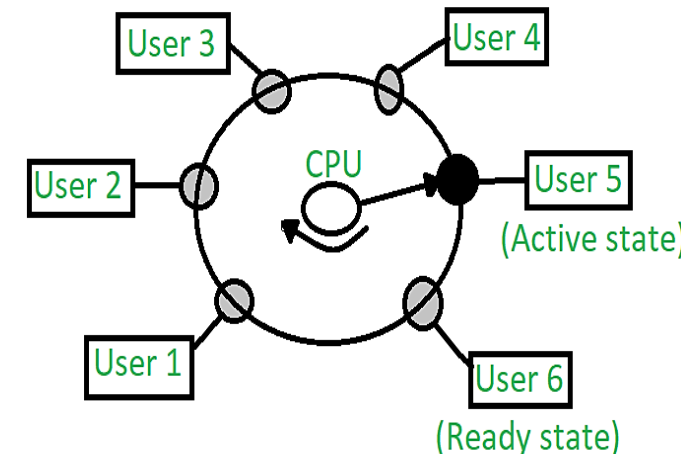
Multiprogramming



- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing

- **Response time** should be < 1 second
- Each user has at least one program executing in memory $\Rightarrow$ **process**
- If several jobs ready to run at the same time $\Rightarrow$ **CPU scheduling**
- If processes don't fit in memory, **swapping** moves them in and out to run
- **Virtual memory** allows execution of processes not completely in memory

Time Sharing



Operating System functions

- The main functions of operating systems are:
 1. Program creation
 2. Program execution
 3. Input/Output operations
 4. Error detection
 5. Resource allocation
 6. Accounting
 7. protection

Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an [interrupt](#)

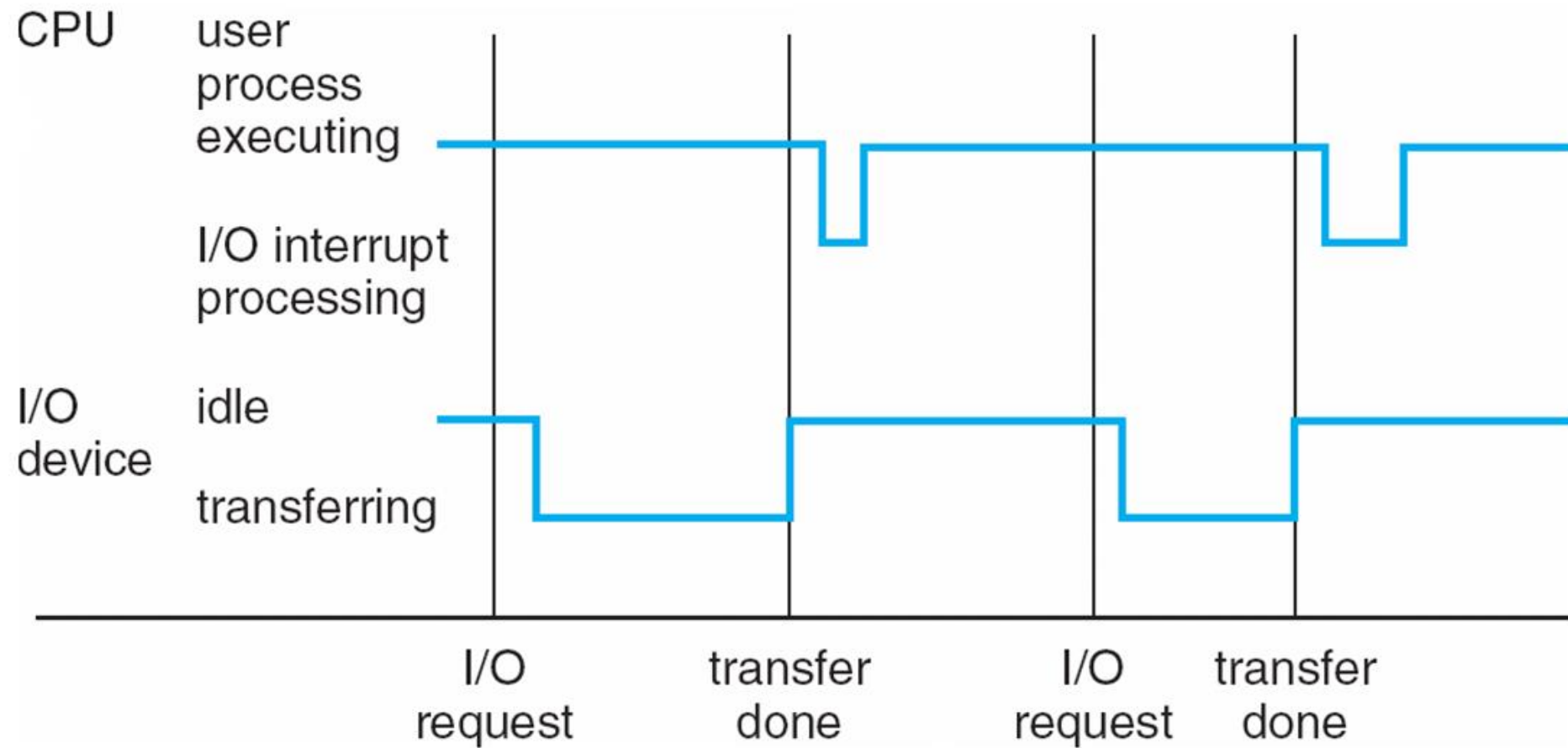
Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**

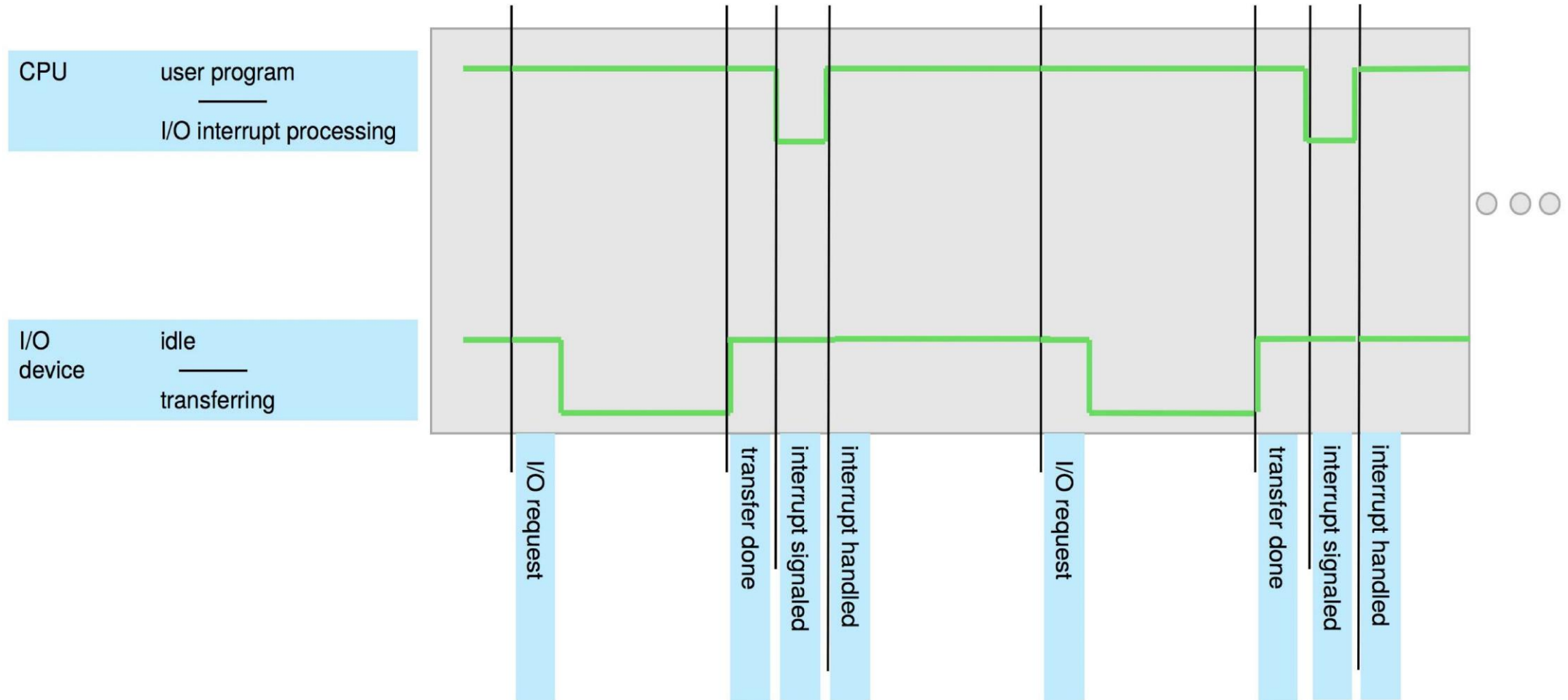
Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
 - **polling**
 - **vectored** interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt

Interrupt Timeline



Interrupt Timeline



I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access)
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
 - **System call** – request to the OS to allow user to wait for I/O completion
 - **Device-status table** contains entry for each I/O device indicating its type, address, and state
 - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt

Storage Definitions and Notation

- The basic unit of computer storage is the **bit**.
- A bit can contain one of two values, 0 and 1.
- A **byte** is 8 bits, and on most computers, it is the smallest convenient chunk of storage.

Unit Conversions

- A **kilobyte**, or **KB**, is 1,024 bytes
- a **megabyte**, or **MB**, is $1,024^2$ bytes
- a **gigabyte**, or **GB**, is $1,024^3$ bytes
- a **terabyte**, or **TB**, is $1,024^4$ bytes
- a **petabyte**, or **PB**, is $1,024^5$ bytes

$$1 \text{ Byte} = 8 \text{ bits}$$

$$1 \text{ Kilobyte (Kb)} = 1,024 \text{ Bytes}$$

$$1 \text{ Megabyte (MB)} = 1,024 \text{ Kilobytes}$$

$$1 \text{ Gigabyte (GB)} = 1,024 \text{ Megabytes}$$

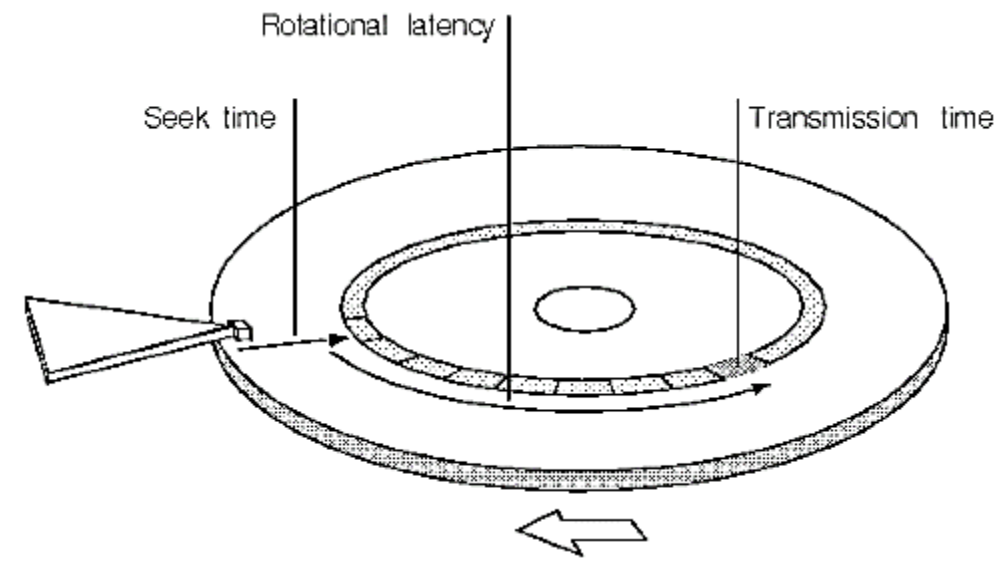
$$1 \text{ Terabyte (TB)} = 1,024 \text{ Gigabytes}$$

$$1 \text{ Petabyte (PB)} = 1,024 \text{ Terabytes}$$

$$1 \text{ Exabyte (EB)} = 1,024 \text{ Petabytes}$$

Storage Structure

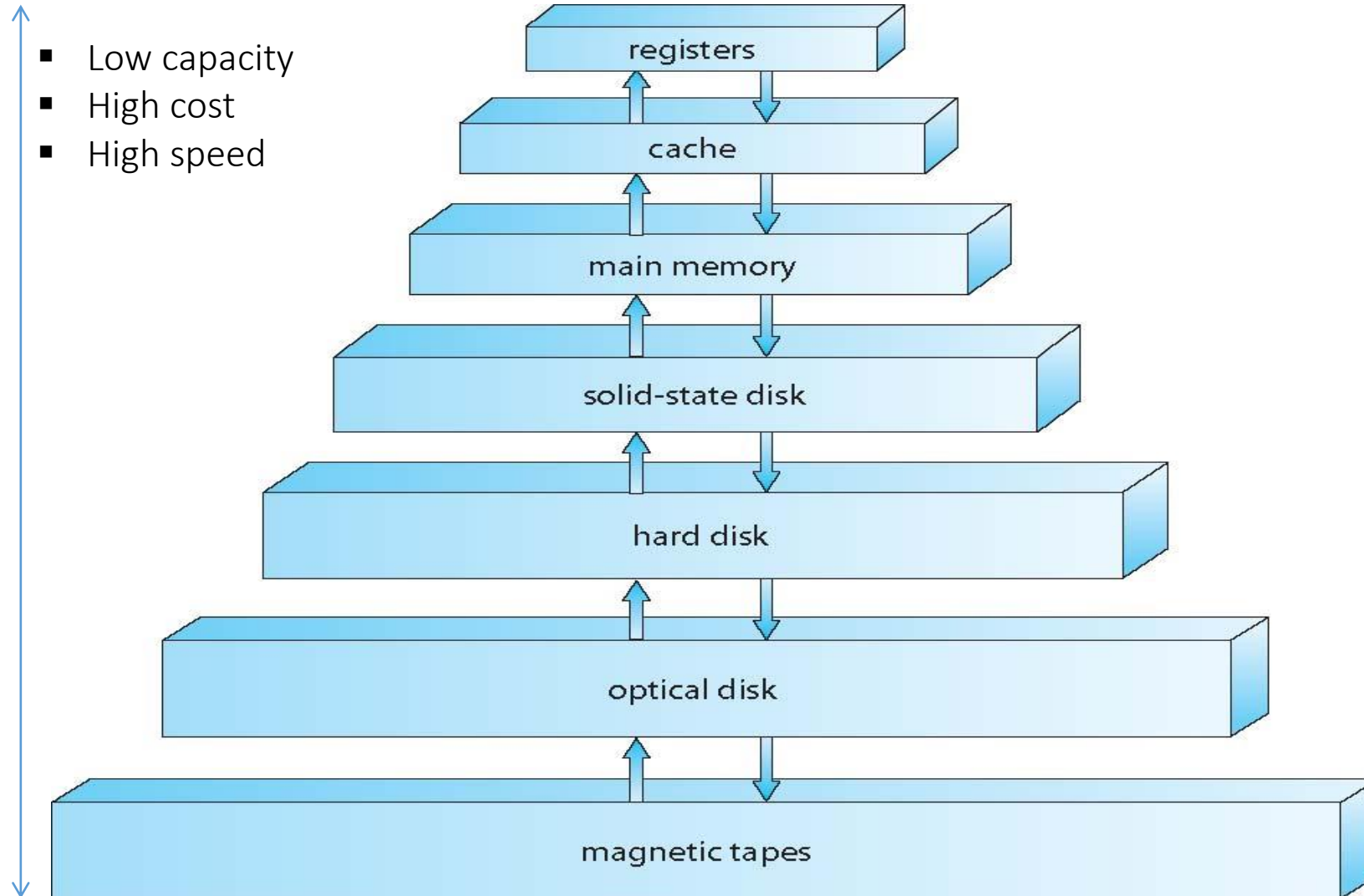
- Main memory – only large storage media that the CPU can access directly
 - **Random access**
 - Typically, **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
 - The **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks** – faster than hard disks, nonvolatile
 - Various technologies
 - Becoming more popular



Storage Hierarchy

- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- **Device Driver** for each device controller to manage I/O
 - Provides uniform interface between controller and kernel

Storage-Device Hierarchy



Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
 - Cache manage important design problem
 - Cache size and replacement policy

Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte

Operating-System Operations

- **Interrupt driven** (hardware and software)
 - Hardware interrupt by one of the devices
 - Software interrupt (**exception** or **trap**):
 - Software error (e.g., division by zero)
 - Request for operating system service
 - Other process problems include infinite loop, processes modifying each other or the operating system

Operating-System Operations (cont.)

- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode**
 - **Mode bit** provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code
 - Some instructions designated as **privileged**, only executable in kernel mode
 - System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
 - i.e. **virtual machine manager (VMM)** mode for guest **VMs**

Introduction to Linux Shell and Kernel

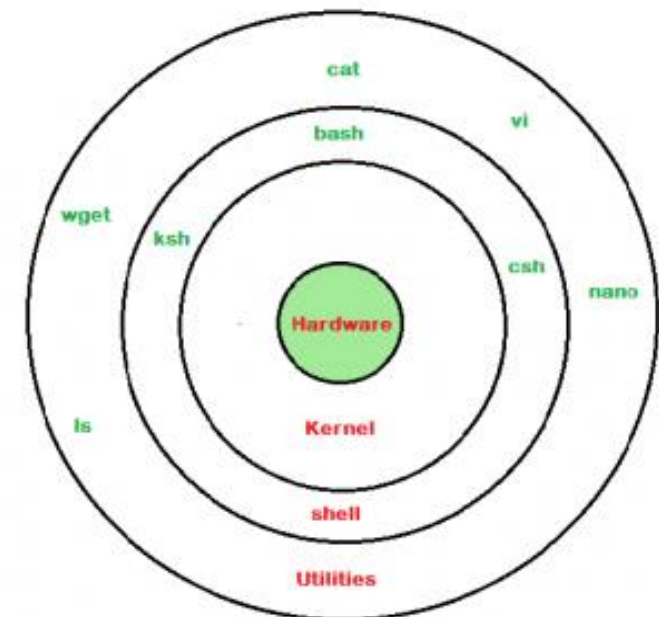
- What is Kernel?
- The kernel is a computer program that is the core of a computer's operating system, with complete control over everything in the system. It manages following resources of the Linux system –
 - File management
 - Process management
 - I/O management
 - Memory management
 - Device management etc
- It is often mistaken that Linus Torvalds has developed Linux OS, but actually he is only responsible for development of Linux kernel.
- Complete Linux system = Kernel + GNU system utilities and libraries + other management scripts + installation scripts.

What is Shell?

- A shell is special user program which provide an interface to user to use operating system services.
- Shell accept **human readable commands** from user and convert them into something which kernel can understand.
- It is a command language interpreter that execute commands read from input devices such as keyboards or from files.
- The shell gets started when the user logs in or start the terminal.

Shell is broadly classified into two categories –

- Command Line Shell
- Graphical shell



KERNEL

A computer program which acts as the core of the computer's operating system and has the control over everything in the system

Core of the system that controls all the tasks of the system

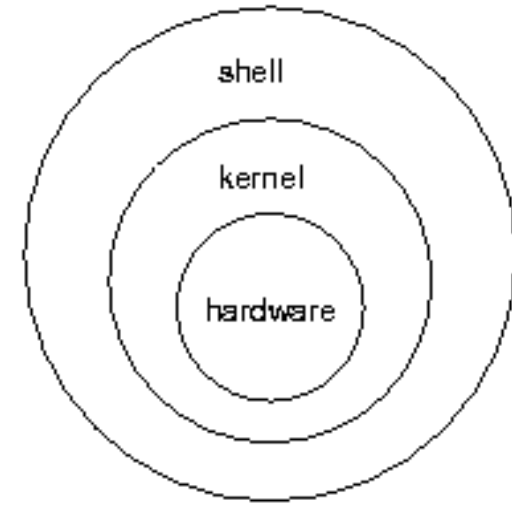
Does not have types

SHELL

A computer program which works as the interface to access the services provided by the operating system

Interface between the kernel and user

Has types such as Bourne shell, C shell, Korn Shell, Bourne Again Shell, etc.



Shell ✓

1. Shell allows the user to communicate with the kernel. ✓

2. Shell is the interface between kernel and user. ✓

3. It is a command line interface.

4. It is the outer layer of Operating System.

Kernal

1. Kernal controls all tasks of the system. ✓

2. It is the core of operating system.

3. Its a low level programme

4. It is the inner layer of O/S.

Thanks